

[↑ Share](#) ...

How to bring one board: provide full path steps

If by “**bring up one board**” you mean **bring a completely bare BeagleBone Black board from power-on to a Linux system capable of running your own C/C++ application**, the full path is:

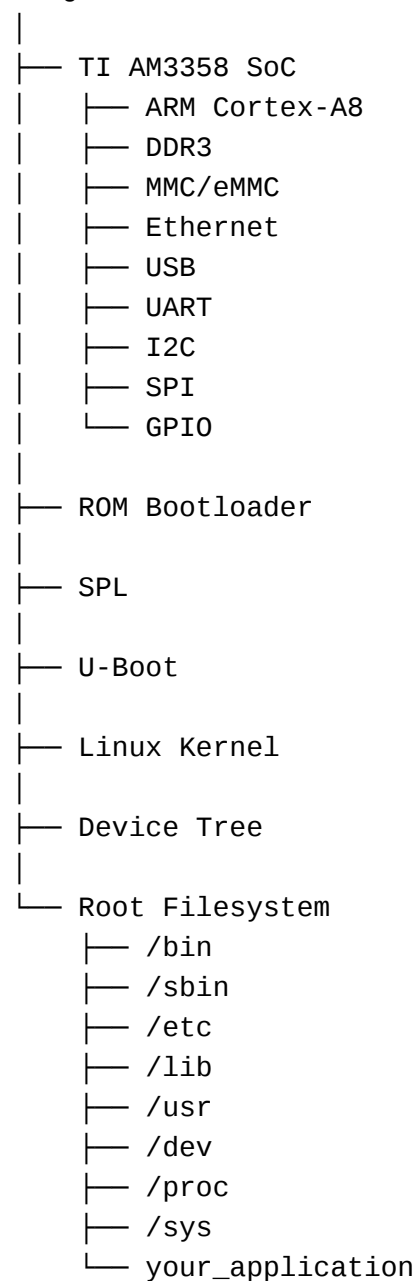
Host PC → Cross Toolchain → U-Boot → Linux Kernel → Root Filesystem → Device Tree → SD/eMMC Boot → Login → Libraries → C/C++ Application

Below is the complete **from-scratch BSP bring-up path**, without Yocto/Buildroot.

1. Target Architecture

For **BeagleBone Black (BBB)**:

BeagleBone Black



The final boot flow is:

```
Power ON
  ↓
AM3358 ROM
  ↓
SPL
  ↓
U-Boot
  ↓
Linux Kernel
  ↓
Device Tree
  ↓
Root Filesystem
  ↓
/sbin/init
  ↓
Your C/C++ Application
```



2. Prepare Host Machine

I recommend using Ubuntu for the development machine.

Create the BSP workspace:

```
</> Bash

mkdir -p ~/bbb-bsp
cd ~/bbb-bsp

mkdir -p sources
mkdir -p toolchain
mkdir -p sysroot
mkdir -p build
mkdir -p rootfs
mkdir -p images
mkdir -p deploy
```



Directory:

```
~/bbb-bsp/  
├─ sources/  
├─ toolchain/  
├─ sysroot/  
├─ build/  
├─ rootfs/  
├─ images/  
└─ deploy/
```



3. Install Host Dependencies

```
</> Bash  
  
sudo apt update
```



Install basic development tools:

```
</> Bash  
  
sudo apt install \  
    build-essential \  
    git \  
    wget \  
    curl \  
    bc \  
    bison \  
    flex \  
    libssl-dev \  
    libncurses-dev \  
    device-tree-compiler \  
    u-boot-tools \  
    xz-utils \  
    bzip2 \  
    cpio \  
    rsync \  
    python3 \  
    python3-pip \  
    unzip \  
    dosfstools \  
    e2fsprogs \  
    parted \  
    gcc-arm-linux-gnueabihf \  
    g++-arm-linux-gnueabihf
```



Check compiler:

```
</> Bash
```



```
arm-linux-gnueabi-hf-gcc --version
```

Check architecture:

```
</> Bash
```



```
arm-linux-gnueabi-hf-gcc -dumpmachine
```

Expected:

```
arm-linux-gnueabi-hf
```



4. Understand the Toolchain

Your BBB uses a **32-bit ARM Cortex-A8**.

Therefore:

```
Host:
```

```
x86_64
```



```
Target:
```

```
ARM 32-bit
```

```
Compiler:
```

```
arm-linux-gnueabi-hf-gcc
```

```
C++:
```

```
arm-linux-gnueabi-hf-g++
```

Set:

```
</> Bash
```



```
export ARCH=arm
```

```
export CROSS_COMPILE=arm-linux-gnueabi-hf-
```

You can add them permanently:

```
</> Bash
```



```
nano ~/.bashrc
```

Add:

```
</> Bash
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
```

Then:

```
</> Bash
source ~/.bashrc
```

5. Build Your Own Toolchain

There are two different approaches.

Approach A — Use Ubuntu's existing cross compiler

This is the easiest:

```
</> Bash
arm-linux-gnueabihf-gcc
```

Approach B — Build GCC/binutils/glibc yourself

Since you want to understand **full BSP bring-up from scratch**, eventually learn:

```
binutils
  ↓
Linux headers
  ↓
GCC bootstrap
  ↓
glibc
  ↓
GCC final
```

The resulting toolchain looks like:

```
~/bbb-bsp/toolchain/
├─ bin/
│   ├── arm-linux-gnueabi-hf-gcc
│   ├── arm-linux-gnueabi-hf-g++
│   ├── arm-linux-gnueabi-hf-ld
│   ├── arm-linux-gnueabi-hf-as
│   └── ...
├─ arm-linux-gnueabi-hf/
└─ lib/
```

For your **first board bring-up**, however, use the existing Ubuntu cross compiler. Build a custom toolchain after the board boots successfully.

6. Obtain U-Boot Source

Download U-Boot:

```
</> Bash
cd ~/bbb-bsp/sources
git clone https://source.denx.de/u-boot/u-boot.git
```

Enter:

```
</> Bash
cd u-boot
```

For a reproducible BSP, select a known release/tag rather than always using the moving master branch.

For example:

```
</> Bash
git tag
```

Then:

```
</> Bash
git checkout <known-release>
```

7. Configure U-Boot for BBB

Clean:

```
</> Bash  
  
make distclean
```



Configure:

```
</> Bash  
  
make am335x_evm_defconfig
```



This configuration is used for AM335x boards and is the starting point for BBB support.

Build:

```
</> Bash  
  
make -j$(nproc)
```



Important output:

```
MLO  
u-boot.img
```



Depending on the U-Boot version/configuration, additional artifacts may be produced.

8. Understand MLO/SPL

BBB boot does not simply execute `u-boot.img`.

The boot chain is roughly:

```
AM3358 ROM  
  ↓  
SPL / MLO  
  ↓  
U-Boot  
  ↓  
Linux
```



SPL's job includes early initialization such as:

```
CPU initialization
  ↓
clock initialization
  ↓
DDR initialization
  ↓
load full U-Boot
```



So:

```
MLO = SPL boot image
u-boot.img = full U-Boot
```



9. Build Linux Kernel

Download Linux:

```
</> Bash
cd ~/bbb-bsp/sources
git clone https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
```



Enter:

```
</> Bash
cd linux
```



Select a known stable release.

Then:

```
</> Bash
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
```



Configure:

```
</> Bash
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- multi_v7_defconfig
```



For BBB-specific configuration, inspect the available configuration:

```
</> Bash
```



```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
```

Make sure you have support for:

```
AM33xx SoC
ARM Cortex-A8
MMC
EXT4
FAT
USB
Ethernet
Serial
GPIO
I2C
SPI
Device Tree
```



10. Build Kernel

```
</> Bash
```



```
make -j$(nproc) \
    ARCH=arm \
    CROSS_COMPILE=arm-linux-gnueabihf-
```

Build device trees:

```
</> Bash
```



```
make -j$(nproc) \
    ARCH=arm \
    CROSS_COMPILE=arm-linux-gnueabihf- \
    dtbs
```

Kernel image:

```
arch/arm/boot/zImage
```



BBB device tree will be under:

```
arch/arm/boot/dts/
```



The exact filename can vary by kernel release, so inspect:

```
</> Bash
```



```
ls arch/arm/boot/dts/*am335*
```

11. Understand Device Tree

Device Tree tells Linux:

"This hardware exists, and this is how it is connected."

For example:

```
CPU
```



```
|— UART
|— MMC
|— Ethernet
|— I2C
|— SPI
|— GPIO
```

Without the correct hardware description, your kernel may boot but peripherals may not work.

The important file is the BBB .dtb .

Conceptually:

```
am335x-boneblack.dts
```



```
↓
```

```
dtc
```

```
↓
```

```
am335x-boneblack.dtb
```

The .dtb is passed to Linux by U-Boot.

12. Build Device Tree Manually

You can inspect:

```
</> Bash  
  
cd ~/bbb-bsp/sources/linux
```



Then:

```
</> Bash  
  
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- dtbs
```



Find output:

```
</> Bash  
  
find arch/arm/boot/dts -name "*bone*"
```



13. Root Filesystem

Now comes the most important part.

The kernel alone is **not a complete Linux system**.

You need:

```
Kernel  
+  
Device Tree  
+  
Root Filesystem
```



Your root filesystem should contain:

```
rootfs/
├─ bin/
├─ sbin/
├─ lib/
├─ usr/
│   ├─ bin/
│   ├─ lib/
│   └─ sbin/
├─ etc/
├─ dev/
├─ proc/
├─ sys/
├─ run/
├─ tmp/
├─ home/
├─ root/
├─ var/
└─ opt/
```



14. Build Root Filesystem Without Yocto

For a pure from-scratch educational BSP, you can build the userspace yourself.

You need at minimum:

```
BusyBox
glibc
dynamic loader
libraries
/etc
/dev
/proc
/sys
init
```



Download BusyBox:

```
</> Bash

cd ~/bbb-bsp/sources

git clone https://git.busybox.net/busybox
```



Build:

```
</> Bash  
  
cd busybox  
  
make distclean
```



Configure:

```
</> Bash  
  
make defconfig
```



Then:

```
</> Bash  
  
make menuconfig
```



Enable:

```
Settings  
→  
    Build static binary
```



For the first bring-up, a **static BusyBox** makes life much easier because you don't initially need to solve all dynamic library dependencies.

Build:

```
</> Bash  
  
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- \  
-j$(nproc)
```



Install into your rootfs:

```
</> Bash  
  
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- \  
CONFIG_PREFIX=~/.bbb-bsp/rootfs \  
install
```



15. Create Root Filesystem Directories

```
</> Bash

cd ~/bbb-bsp/rootfs

mkdir -p \
  dev \
  proc \
  sys \
  run \
  tmp \
  etc \
  home \
  root \
  var \
  opt \
  mnt \
  lib \
  usr/lib
```



16. Create /etc/inittab

```
</> Bash

nano ~/bbb-bsp/rootfs/etc/inittab
```



Example:

```
::sysinit:/etc/init.d/rcS
ttyS0::askfirst:~/bin/sh
::restart:/sbin/init
::ctrlaltdel:/sbin/reboot
```



The serial device name can depend on the kernel/device-tree configuration.

17. Create Init Script

```
</> Bash

mkdir -p ~/bbb-bsp/rootfs/etc/init.d
```



Create:

```
</> Bash

nano ~/bbb-bsp/rootfs/etc/init.d/rcS
```



Example:

```
</> Bash

#!/bin/sh

mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev

echo "=====
echo " BeagleBone Black Linux Started"
echo "=====

hostname bbb
```



Make executable:

```
</> Bash

chmod +x ~/bbb-bsp/rootfs/etc/init.d/rcS
```



18. Device Files

For the first boot, enable kernel:

```
CONFIG_DEVTMPFS=y
CONFIG_DEVTMPFS_MOUNT=y
```



Then Linux automatically creates:

```
/dev/ttyS0
/dev/mmcblk0
/dev/null
/dev/console
...
```



This is much easier than manually creating every device node.

19. Create /etc/fstab

</> Bash

```
nano ~/bbb-bsp/rootfs/etc/fstab
```

Example:

```
proc    /proc      proc      defaults    0 0
sysfs   /sys          sysfs     defaults    0 0
tmpfs   /tmp          tmpfs     defaults    0 0
```

20. Add Your C Application

Create:

</> Bash

```
mkdir -p ~/bbb-bsp/rootfs/opt/myapp
```

Example application:

</> C

```
#include <stdio.h>

int main(void)
{
    printf("Hello from BeagleBone Black!\n");
    return 0;
}
```

Compile:

</> Bash

```
arm-linux-gnueabi-gcc hello.c -o hello
```

Check:

</> Bash

```
file hello
```

You should see ARM architecture information.

Copy:

```
</> Bash  
  
cp hello ~/bbb-bsp/rootfs/opt/myapp/
```



21. Build Dynamic C/C++ Application Later

For a real application you will eventually have:

```
Application  
  ↓  
libc  
  ↓  
libstdc++  
  ↓  
libgcc  
  ↓  
Linux system calls  
  ↓  
Linux kernel  
  ↓  
Hardware
```



For example:

```
</> Bash  
  
arm-linux-gnueabihf-g++ main.cpp \  
-o myapp
```



If dynamically linked, make sure the target contains the required:

```
/lib/ld-linux-armhf.so.3  
/lib/libc.so.6  
/lib/libstdc++.so.6  
/lib/libgcc_s.so.1
```



22. Create SD Card Layout

The BBB boot media generally needs:

SD Card

```
|
├── Boot Partition
│   ├── MLO
│   ├── u-boot.img
│   ├── zImage
│   └── *.dtb
└── Root Filesystem Partition
    ├── bin/
    ├── sbin/
    ├── etc/
    ├── lib/
    ├── usr/
    ├── dev/
    ├── proc/
    ├── sys/
    └── opt/
```



A typical layout:

/dev/sdX

```
|
├── /dev/sdX1    FAT32
│   └── boot files
└── /dev/sdX2    ext4
    └── rootfs
```



Be extremely careful with `/dev/sdX` . Selecting the wrong disk can erase your host machine.

23. Partition SD Card

Identify the SD card:

```
<> Bash
```

```
lsblk
```



Example:

```
sda      477G
├─sda1
└─sda2

sdb      32G
```



Suppose:

```
/dev/sdb
```



is your BBB SD card.

Use:

```
</> Bash
sudo fdisk /dev/sdb
```



Create:

```
Partition 1:
FAT32
~128 MB

Partition 2:
Linux
remaining space
```



Then:

```
</> Bash
sudo partprobe /dev/sdb
```



Format boot:

```
</> Bash
sudo mkfs.vfat -F 32 /dev/sdb1
```



Format rootfs:

```
</> Bash
sudo mkfs.ext4 /dev/sdb2
```



24. Mount SD Card

</> Bash



```
sudo mkdir -p /mnt/bbb-boot  
sudo mkdir -p /mnt/bbb-root
```

Mount:

</> Bash



```
sudo mount /dev/sdb1 /mnt/bbb-boot  
sudo mount /dev/sdb2 /mnt/bbb-root
```

25. Copy Bootloader

From U-Boot build directory:

</> Bash



```
cd ~/bbb-bsp/sources/u-boot
```

Copy SPL/MLO:

</> Bash



```
sudo cp MLO /mnt/bbb-boot/
```

Copy U-Boot:

</> Bash



```
sudo cp u-boot.img /mnt/bbb-boot/
```

26. Copy Linux Kernel

</> Bash



```
sudo cp ~/bbb-bsp/sources/linux/arch/arm/boot/zImage \  
/mnt/bbb-boot/
```

Copy BBB DTB:

```
</> Bash   
  
sudo cp ~/bbb-bsp/sources/linux/arch/arm/boot/dts/<bbb-dtb-file>.dtb \  
/mnt/bbb-boot/
```

Your boot partition should now resemble:

```
/mnt/bbb-boot/   
  
MLO  
u-boot.img  
zImage  
am335x-....dtb
```

27. Copy Root Filesystem

```
</> Bash   
  
sudo cp -a ~/bbb-bsp/rootfs/. /mnt/bbb-root/
```

Fix ownership:

```
</> Bash   
  
sudo chown -R root:root /mnt/bbb-root
```

28. Sync

```
</> Bash   
  
sync
```

Unmount:

```
</> Bash 
```

```
sudo umount /mnt/bbb-boot
sudo umount /mnt/bbb-root
```

Remove SD card safely.

29. Connect Serial Console

This is **very important for board bring-up**.

Do not depend only on HDMI/network initially.

Use a **3.3 V USB-to-UART adapter**.

Connect:

USB-UART		BBB

GND	→	GND
RX	→	UART TX
TX	→	UART RX



Do **not** connect 5 V UART signals.

Open terminal:

```
</> Bash
sudo apt install minicom
```



Then:

```
</> Bash
minicom -D /dev/ttyUSB0 -b 115200
```



Typical BBB serial settings:

Baud:	115200
Data:	8
Parity:	None
Stop:	1
Flow:	None



30. Insert SD Card and Power BBB

Insert the SD card.

Connect serial console.

Power the board.

You should see something similar to:

```
U-Boot SPL ...  
Trying to boot from MMC1
```



Then:

```
U-Boot ...
```



Then Linux:

```
Starting kernel ...
```



Then:

```
[ 0.000000] Booting Linux ...
```



Eventually:

```
BeagleBone Black Linux Started  
/ #
```



That is your first successful board bring-up.

31. Test Linux

At:

```
/ #
```



Run:

```
</> Bash  
  
uname -a
```



Check CPU:

```
</> Bash
```

```
cat /proc/cpuinfo
```



Check memory:

```
</> Bash
```

```
cat /proc/meminfo
```



Check devices:

```
</> Bash
```

```
ls /dev
```



Check kernel:

```
</> Bash
```

```
dmesg
```



Check filesystem:

```
</> Bash
```

```
mount
```



32. Test Your C Application

```
</> Bash
```

```
cd /opt/myapp
```



Run:

```
</> Bash
```

```
./hello
```



Expected:

Hello from BeagleBone Black!



At this point your complete path is working:

Your C Code



↓

ARM Cross Compiler

↓

ARM Executable

↓

Root Filesystem

↓

SD Card

↓

BBB

↓

U-Boot

↓

Linux Kernel

↓

User Space

↓

Your Application

33. Next Bring-Up Stage: Ethernet

Once serial boot works, enable Ethernet.

Check:

⟨⟩ Bash



ip link

You should see an interface such as:

eth0



Bring it up:

⟨⟩ Bash



ip link set eth0 up

Configure IP:

```
</> Bash  
ip addr add 192.168.1.100/24 dev eth0
```



Test:

```
</> Bash  
ping 192.168.1.1
```



Once networking works, you can stop depending on SD-card copying.

34. Enable SSH

For a real BSP:

```
UART  
↓  
Ethernet  
↓  
SSH  
↓  
SCP  
↓  
Application deployment
```



You can add an SSH server such as Dropbear/OpenSSH to your root filesystem.

Then:

```
</> Bash  
ssh root@<BBB-IP>
```



Copy applications:

```
</> Bash  
scp myapp root@<BBB-IP>:/opt/myapp/
```



35. Add Libraries

For a real C/C++ application:

```
/opt/myapp/  
├─ bin/  
│   └─ MyApplication  
├─ lib/  
│   ├── libMyLibrary.so  
│   ├── libfoo.so  
│   └─ libbar.so  
├─ config/  
└─ models/
```



Set:

```
</> Bash
```



```
export LD_LIBRARY_PATH=/opt/myapp/lib:$LD_LIBRARY_PATH
```

Or configure `/etc/ld.so.conf.d/` .

36. Add Application Startup

Eventually you want:

```
Power ON  
  ↓  
Bootloader  
  ↓  
Linux  
  ↓  
init  
  ↓  
network  
  ↓  
application
```



For example:

```
/etc/init.d/  
  ↓  
myapp  
  ↓  
/opt/myapp/MyApplication
```



Then your application starts automatically after Linux boots.

37. Full BSP Development Path

Once the basic board works, develop the BSP in this order:

PHASE 1

Host machine



Cross compiler



PHASE 2

Bootloader



SPL



U-Boot



PHASE 3

Linux



Kernel configuration



Kernel build



PHASE 4

Device Tree



CPU



UART



MMC



Ethernet



GPIO



I2C



SPI



USB



PHASE 5
Root filesystem
↓
BusyBox
↓
init
↓
devtmpfs
↓
proc
↓
sysfs



PHASE 6
Storage
↓
SD
↓
eMMC



PHASE 7
Networking
↓
Ethernet
↓
DHCP/static IP
↓
SSH



PHASE 8
Application
↓
C
↓
C++
↓
Libraries
↓
Configuration
↓
Startup service



38. What You Should NOT Do Initially

For the first successful board bring-up, don't try to implement everything simultaneously.

Don't start with:

```
Kernel
+ custom GCC
+ custom glibc
+ custom U-Boot
+ custom Device Tree
+ Ethernet
+ USB
+ GUI
+ Qt
+ application
```



Instead:

```
Cross compiler
  ↓
U-Boot
  ↓
Kernel
  ↓
Device Tree
  ↓
BusyBox rootfs
  ↓
Serial console
  ↓
C application
  ↓
Ethernet
  ↓
SSH
  ↓
Drivers
  ↓
Production BSP
```



This gives you a clear debugging boundary at every stage.

39. Final Production BSP Structure

I recommend eventually organizing your project like this:

bbb-bsp/



```
|
├── sources/
│   ├── u-boot/
│   ├── linux/
│   ├── busybox/
│   ├── gcc/
│   ├── binutils/
│   └── glibc/
|
├── toolchain/
|
├── sysroot/
|
├── kernel/
│   ├── config/
│   └── patches/
|
├── uboot/
│   ├── config/
│   └── patches/
|
├── device-tree/
│   ├── board.dts
│   └── patches/
|
├── rootfs/
│   ├── bin/
│   ├── sbin/
│   ├── etc/
│   ├── lib/
│   ├── usr/
│   ├── dev/
│   ├── proc/
│   ├── sys/
│   └── opt/
|
├── application/
│   ├── src/
│   ├── include/
│   └── Makefile
|
├── scripts/
│   ├── build-uboot.sh
│   ├── build-kernel.sh
│   ├── build-rootfs.sh
│   ├── make-sd.sh
│   └── deploy.sh
|
└── images/
```



```
|   |— MLO  
|   |— u-boot.img  
|   |— zImage  
|   |— board.dtb  
|   |— rootfs.ext4  
|  
└─ deploy/
```

This is the **manual BSP approach** you were asking about: no Yocto, no BitBake, and no Buildroot. The important thing is that you understand and can independently build each layer.

The most practical next step is to do this specifically for your BeagleBone Black one command at a time, starting from `~/bbb-bsp`, and build **U-Boot** → **Linux** → **BusyBox** → **rootfs** → **SD card** → **serial boot**.



How to bring one board: provide full path steps
not only particular board:
Any hardware board if we want to build then?

Yes. If your goal is to learn **how to bring up ANY embedded Linux hardware board from scratch**, you should think in terms of a **generic BSP/board bring-up methodology**, not BeagleBone-specific commands.

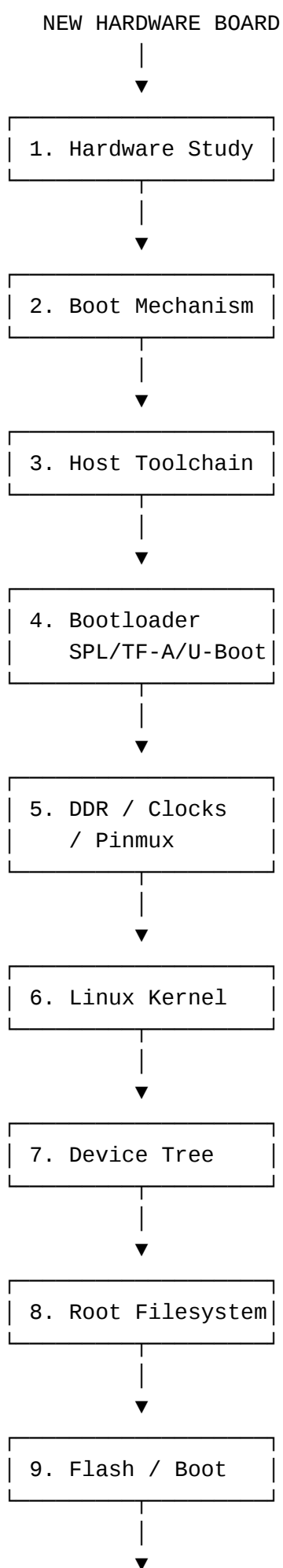
The board could be:

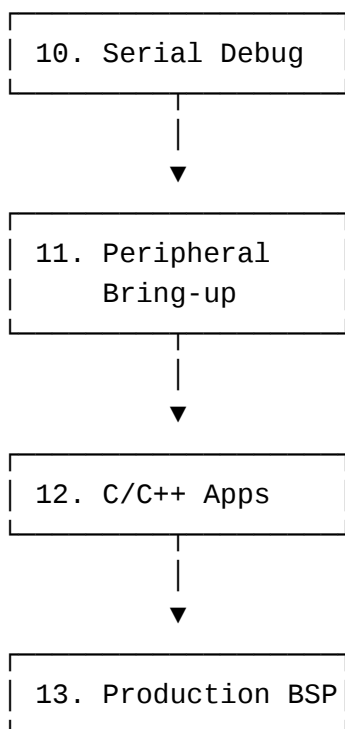
- BeagleBone Black
- Raspberry Pi
- STM32MP1
- NXP i.MX6/i.MX8
- TI AM335x/AM62x
- Rockchip
- Amlogic
- Qualcomm
- Allwinner
- Custom company board

The exact bootloader, kernel, device tree, firmware and flashing mechanism change, but the **bring-up path is largely the same**.

Generic Embedded Linux Board Bring-Up Path

Think of the entire process as:





1. Start With Hardware, Not Linux

Before touching the kernel, understand the board.

Collect:

- SoC datasheet
- SoC reference manual
- Board schematic
- PCB documentation
- BOM
- Memory datasheet
- PMIC datasheet
- Ethernet PHY datasheet
- USB controller/PHY information
- Wi-Fi/BT module information
- Display information
- Camera information
- Boot-mode information



The first question should be:

What SoC is on this board?

For example:

Board
↓
Amlogic A311D



or:

Board
↓
TI AM3358



or:

Board
↓
NXP i.MX8



The **SoC determines most of your BSP architecture.**

2. Identify the CPU Architecture

From the SoC, determine:

Architecture
↓
ARM / ARM64 / RISC-V / x86



Examples:

SoC	Architecture
AM3358	ARM32 Cortex-A8
i.MX6	ARM32
i.MX8	ARM64
A311D	ARM64
RK3568	ARM64
STM32MP1	ARM32
RISC-V SoC	RISC-V

Then determine:

32-bit or 64-bit



This determines your compiler.

For ARM32:

`</> Bash`



`arm-linux-gnueabi-hf-gcc`

For ARM64:

`</> Bash`



`aarch64-linux-gnu-gcc`

For RISC-V:

`</> Bash`



`riscv64-linux-gnu-gcc`

3. Understand the Boot ROM

Every modern SoC has some form of immutable boot code.

Conceptually:

Power ON



SoC Boot ROM



Read boot configuration



Select boot device



Load first-stage firmware



Boot device might be:

SD
eMMC
SPI NOR
NAND
USB
UART
Network



The exact mechanism depends on the SoC.

This is one of the first things you must determine from the SoC documentation.

4. Understand the Boot Chain

Do not assume every board uses exactly:

ROM → SPL → U-Boot → Linux



Different SoCs have different boot chains.

For example, you may encounter:

ROM
↓
SPL
↓
U-Boot
↓
Linux



or:

ROM
↓
TF-A
↓
U-Boot
↓
Linux



or:

```
ROM
↓
SPL
↓
TF-A
↓
U-Boot
↓
Linux
```



or vendor-specific firmware:

```
ROM
↓
Vendor BL1
↓
Vendor BL2
↓
ATF
↓
U-Boot
↓
Linux
```



Therefore the generic question is:

What firmware does this SoC require between ROM and Linux?

5. Determine the Boot Media

Find out where your board will boot from.

Common options:

```
SD Card
eMMC
SPI NOR
NAND
USB
Network
```



For development, SD or USB boot is often convenient if the hardware supports it.

For production:

SPI/eMMC/NAND



may be used.

You need to understand the complete storage layout.

For example:

eMMC



```
|  
├── Bootloader  
├── Boot partition  
└── Root filesystem
```

or:

SPI NOR



```
|  
└── Bootloader
```

eMMC

```
|  
├── Kernel  
├── DTB  
└── RootFS
```

6. Create a Generic BSP Workspace

On your Ubuntu development machine:

</> Bash



```
mkdir -p ~/board-bsp  
cd ~/board-bsp
```

Create:

</> Bash



```
mkdir -p \  
  sources \  
  toolchain \  
  sysroot \  
  bootloader \  
  kernel \  
  device-tree \  
  rootfs \  
  application \  
  images \  
  scripts \  
  deploy \  
  docs
```

You now have:

```
board-bsp/  
|  
├─ sources/  
├─ toolchain/  
├─ sysroot/  
├─ bootloader/  
├─ kernel/  
├─ device-tree/  
├─ rootfs/  
├─ application/  
├─ images/  
├─ scripts/  
├─ deploy/  
└─ docs/
```



This structure can be used for almost any Linux-capable board.

7. Select the Cross Compiler

Determine:

```
CPU architecture  
ABI  
floating point ABI  
C library  
compiler version
```



For example:

```
Architecture = ARM32
ABI           = hard-float
Target        = arm-linux-gnueabihf
```



Then:

```
</> Bash

export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
```



For ARM64:

```
</> Bash

export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-
```



For RISC-V:

```
</> Bash

export ARCH=riscv
export CROSS_COMPILE=riscv64-linux-gnu-
```



The important point is:

Never blindly select a compiler because the board is called "ARM".

You need to know the actual architecture and ABI.

8. Build or Obtain Toolchain

The toolchain contains:

```
binutils
GCC
glibc/musl
libgcc
libstdc++
GDB
```



Conceptually:

Source code



GCC



Assembler



Linker



ARM/ARM64/RISC-V executable



For example:

⟨⟩ Bash

```
${CROSS_COMPILE}gcc main.c -o myapp
```



Verify:

⟨⟩ Bash

```
file myapp
```



9. Obtain Bootloader Source

Most Linux boards use one or more of:

U-Boot

TF-A

SPL

Vendor boot firmware



Determine which one your SoC requires.

For U-Boot:

⟨⟩ Bash

```
cd ~/board-bsp/sources
git clone https://source.denx.de/u-boot/u-boot.git
```



Then identify the board configuration:

⟨⟩ Bash



```
make list-defconfig
```

Look for:

```
vendor_soc_board_defconfig
```



Configure:

```
</> Bash
```



```
make <board_defconfig>
```

Build:

```
</> Bash
```



```
make -j$(nproc)
```

10. Understand the Bootloader's Job

The bootloader is responsible for preparing the machine for Linux.

It may initialize:

```
CPU
DDR
clocks
pinmux
PMIC
storage
USB
Ethernet
serial
secure world
```



Then it loads:

```
Kernel
DTB
initramfs (optional)
```



into RAM.

Finally:

```
bootloader
  ↓
Linux kernel
```



11. DDR Is a Critical Bring-Up Point

This is especially important for a **new/custom board**.

Linux cannot run properly if DDR isn't initialized.

The chain may be:

```
ROM
  ↓
First-stage bootloader
  ↓
DDR initialization
  ↓
RAM available
  ↓
Full bootloader
```



If DDR initialization fails:

```
No U-Boot
No Linux
No rootfs
```



You may see:

```
nothing on serial
```



or:

```
DDR training failed
```



Therefore, on a custom board, **DDR configuration is often one of the earliest hardware-specific BSP tasks.**

12. Configure Pin Multiplexing

Modern SoCs have multiplexed pins.

One pin might support:

```
GPIO
UART
SPI
I2C
PWM
```



You must configure the appropriate function.

Example:

```
Physical pin
  ↓
Pin controller
  ↓
UART TX
```



or:

```
Physical pin
  ↓
Pin controller
  ↓
SPI MOSI
```



Pinmux may be configured in:

```
Bootloader
Device Tree
Firmware
```



depending on the platform.

13. Build Linux Kernel

Obtain Linux:

```
</> Bash
```



```
cd ~/board-bsp/sources
```

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
```

Configure according to architecture:

```
</> Bash
```

```
make ARCH=arm <board_defconfig>
```

or:

```
</> Bash
```

```
make ARCH=arm64 <board_defconfig>
```

Then:

```
</> Bash
```

```
make ARCH=$ARCH \  
    CROSS_COMPILE=$CROSS_COMPILE \  
    menuconfig
```

Enable required:

```
CPU support  
SoC support  
Memory  
MMC/eMMC  
SPI  
I2C  
GPIO  
USB  
Ethernet  
Networking  
Filesystem  
Serial  
Device Tree
```

Build:

```
</> Bash
```

```
make -j$(nproc) \  
    ARCH=$ARCH \  
    CROSS_COMPILE=$CROSS_COMPILE
```


Build DTBs:

 Bash 

```
make -j$(nproc) \  
    ARCH=$ARCH \  
    CROSS_COMPILE=$CROSS_COMPILE \  
    dtbs
```

14. Device Tree Is the Board Description

This is one of the most important concepts in modern Linux BSP development.

The kernel contains generic driver code.

Device Tree tells the kernel:

```
Which hardware exists?  
Where is it?  
What address does it use?  
Which GPIO?  
Which IRQ?  
Which clock?  
Which regulator?  
Which pinmux?  
Which bus?
```



For example:

Board



```
|
├── UART
│   └── tty
├── I2C
│   └── sensor
├── SPI
│   └── flash
├── MMC
│   └── eMMC
├── Ethernet
│   └── PHY
└── USB
```

The Device Tree represents this hardware topology.

15. Create/Modify Board Device Tree

A typical structure:

SoC DTS



```
|
├── CPU
├── memory
├── clocks
├── pinctrl
├── UART
├── MMC
├── I2C
├── SPI
├── Ethernet
└── USB
```

Your board DTS usually builds on a SoC/common DTS.

Conceptually:

```
soc.dtsi
+
board.dts
↓
board.dtb
```



Compile:

```
</> Bash
dtc ...
```



or preferably through the kernel build system:

```
</> Bash
make dtbs
```



16. Root Filesystem

Linux kernel is not enough.

You need:

```
Kernel
+
DTB
+
RootFS
```



The rootfs contains:

```
/bin
/sbin
/etc
/lib
/usr
/dev
/proc
/sys
/run
/tmp
/home
/opt
```



17. Build RootFS

Without Yocto/Buildroot, you can manually assemble it.

A common minimal solution is:

```
BusyBox
+
glibc
+
libraries
+
init
+
configuration
```



Build BusyBox:

```
</> Bash

cd ~/board-bsp/sources
git clone https://git.busybox.net/busybox
```



Configure:

```
</> Bash

cd busybox
make defconfig
make menuconfig
```



Build:

```
</> Bash

make ARCH=$ARCH \
    CROSS_COMPILE=$CROSS_COMPILE \
    -j$(nproc)
```



Install:

```
</> Bash
```



```
make ARCH=$ARCH \
    CROSS_COMPILE=$CROSS_COMPILE \
    CONFIG_PREFIX=~/.board-bsp/rootfs \
    install
```

18. Add C Library

For dynamically linked applications, your rootfs needs the target C library.

For glibc:

```
/lib/ld-linux-*.so.*
/lib/libc.so.6
/lib/libm.so.6
/lib/libpthread.so.*
/lib/libdl.so.*
```



Exact files depend on architecture and glibc version.

This is where your:

```
sysroot
```



becomes important.

19. Sysroot

The sysroot represents the target's development environment.

Conceptually:

```
sysroot/
├─ usr/
│   ├─ include/
│   └─ lib/
├─ lib/
└─ ...
```



Then compile:

```
</> Bash
```



```
${CROSS_COMPILE}gcc \  
  --sysroot=$HOME/board-bsp/sysroot \  
  main.c \  
  -o myapp
```

For C++:

```
</> Bash  
  
${CROSS_COMPILE}g++ \  
  --sysroot=$HOME/board-bsp/sysroot \  
  main.cpp \  
  -o myapp
```

20. Add Init System

The first userspace process is:

```
PID 1
```

Common choices:

```
BusyBox init  
systemd  
SysV init  
OpenRC  
custom init
```

For a learning BSP:

```
BusyBox init
```

is a good starting point.

Boot:

```
Kernel
↓
/sbin/init
↓
mount filesystems
↓
start services
↓
login shell
```



21. Build the Boot Image

The final image depends heavily on the SoC.

Conceptually:

```
images/
|
├─ bootloader.bin
├─ kernel
├─ board.dtb
├─ initramfs (optional)
└─ rootfs
```



Some boards require:

```
boot.img
```



Some:

```
u-boot.itb
```



Some:

```
Image
dtb
rootfs.ext4
```



Some use:

```
FIT image
```



Some use vendor-specific image formats.

Therefore:

Do not assume every board uses MLO + u-boot.img + zImage .

22. Flash the Board

Possible methods:

SD card
eMMC
SPI NOR
NAND
USB recovery
Fastboot
DFU
JTAG
Network/TFTP
Vendor flashing tool



Development flow might be:

Host PC
↓
SD Card
↓
Board



or:

Host PC
↓ USB
Boot ROM / recovery
↓
eMMC



or:

Host PC
↓ Ethernet
TFTP
↓
U-Boot
↓
RAM



23. First Boot Must Be Serial

For a new board, **serial console is your most important debugging interface.**

Connect:

USB-UART



Board UART



Then:

```
</> Bash
```

```
minicom -D /dev/ttyUSB0 -b 115200
```



Depending on the board:

115200

8N1



may be used, but always verify the SoC/board documentation.

24. Debug Boot in Layers

Never debug the entire system at once.

Use this sequence:

LEVEL 1

Power



Does the board power correctly?



LEVEL 2

ROM



Does the SoC enter the expected boot mode?



LEVEL 3
First-stage firmware



Does SPL/TF-A/vendor firmware execute?



LEVEL 4
DDR



Is RAM initialized?



LEVEL 5
U-Boot



Do you get:

U-Boot #



?



LEVEL 6
Linux



Do you get:

Starting kernel...



?



LEVEL 7
RootFS



Do you get:

/ #



?

↓

LEVEL 8
Peripherals



Can you access:

UART
I2C
SPI
GPIO
MMC
USB
Ethernet



?

↓

LEVEL 9
Application



Can your C/C++ program execute?

25. Peripheral Bring-Up Order

Don't enable every peripheral simultaneously.

A good order is:

1. UART
2. RAM
3. Storage
4. GPIO
5. I2C
6. SPI
7. Ethernet
8. USB
9. RTC
10. PWM
11. ADC
12. Display
13. Camera
14. Audio
15. Wi-Fi/Bluetooth



Why UART first?

Because without serial logs, debugging everything else becomes much harder.

26. Test Each Peripheral Independently

GPIO

```
</> Bash  
  
gpiodetect  
gpioinfo
```



I2C

```
</> Bash  
  
i2cdetect -l  
i2cdetect -y <bus>
```



SPI

Use the appropriate SPI test utility/device.

MMC

```
</> Bash
```



```
ls /dev/mmc*  
dmesg | grep -i mmc
```

Ethernet

```
</> Bash  
  
ip link  
ip addr
```



USB

```
</> Bash  
  
lsusb  
dmesg | grep -i usb
```



27. Bring Up Networking

First:

```
</> Bash  
  
ip link
```



Then:

```
</> Bash  
  
ip link set eth0 up
```



DHCP:

```
</> Bash  
  
udhcpc -i eth0
```



or your chosen network manager.

Test:

```
</> Bash
```



```
ping <gateway>
```

Then:

```
</> Bash
```

```
ping 8.8.8.8
```

Once networking works:

```
UART
```

```
↓
```

```
Ethernet
```

```
↓
```

```
SSH
```

```
↓
```

```
Remote development
```

28. Bring Up Application Environment

Now your target should have:

```
Compiler on host
```

```
↓
```

```
sysroot
```

```
↓
```

```
C/C++ application
```

```
↓
```

```
libraries
```

```
↓
```

```
rootfs
```

```
↓
```

```
board
```

Example:

```
</> Bash
```

```
${CROSS_COMPILE}g++ \  
  --sysroot=$HOME/board-bsp/sysroot \  
  main.cpp \  
  -o MyApplication
```

Copy:

```
</> Bash

scp MyApplication root@BOARD_IP:/opt/
```



Run:

```
</> Bash

/opt/MyApplication
```



29. Shared Libraries

For a production application:

```
MyApplication
├─ libc.so
├─ libstdc++.so
├─ libgcc_s.so
├─ libpthread.so
└─ your custom .so
```



Check dependencies:

```
</> Bash

${CROSS_COMPILE}readelf -d MyApplication
```



On the target:

```
</> Bash

ldd MyApplication
```



if ldd is available.

30. Application Startup

Eventually:

```
Power ON
  ↓
Bootloader
  ↓
Kernel
  ↓
init
  ↓
mount rootfs
  ↓
network
  ↓
drivers
  ↓
application
```



Your application might be:

```
/opt/MyApplication/MyApplication
```



and started by:

```
systemd
```



or:

```
/etc/init.d/
```



depending on your init system.

31. Production BSP

Once everything works, freeze your BSP.

Your production source tree should contain:


```
board-bsp/  
├── bootloader/  
│   ├── source  
│   ├── config  
│   └── patches  
├── kernel/  
│   ├── source  
│   ├── config  
│   └── patches  
├── device-tree/  
│   ├── board.dts  
│   └── patches  
├── toolchain/  
├── sysroot/  
├── rootfs/  
├── application/  
├── firmware/  
├── scripts/  
├── images/  
└── documentation/
```



Record exact versions:

```
SoC  
Board revision  
Bootloader version  
TF-A version  
Linux version  
GCC version  
glibc version  
BusyBox version  
DTB version  
Firmware versions  
Application version
```



This is essential for reproducibility.

32. What Changes From Board to Board?

This is the most important thing to understand.

The **process stays similar**, but these parts change:

Component	Board dependent?
CPU architecture	Yes
Cross compiler	Yes
Boot ROM	Yes
SPL/TF-A	Yes
U-Boot configuration	Yes
DDR initialization	Very much
PMIC	Yes
Pinmux	Yes
Device Tree	Very much
Kernel drivers	Yes
Storage layout	Yes
Flashing method	Yes
Rootfs	Mostly reusable
Application	Mostly independent
Linux concepts	Mostly reusable

So when you receive a **new board**, don't start by searching for random commands.

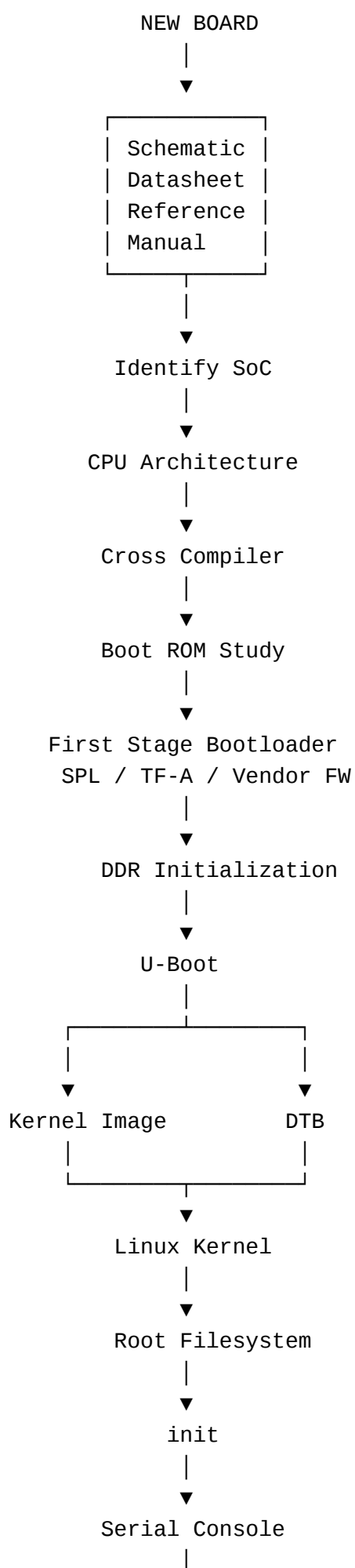
Ask these questions first:

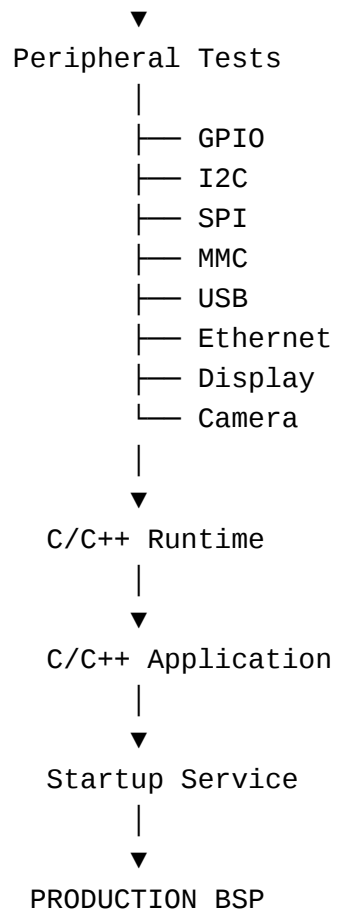
1. What SoC?
2. What CPU architecture?
3. What RAM?
4. What storage?
5. What boot device?
6. What boot ROM expects?
7. What first-stage bootloader?
8. Is TF-A required?
9. Is U-Boot used?
10. Which Linux kernel?
11. Which Device Tree?
12. Which PMIC?
13. Which clock configuration?
14. Which pinmux?
15. Which UART is debug console?
16. How is the board flashed?
17. Which peripherals are required?
18. Which root filesystem?
19. Which C library?
20. Which application/runtime is required?



33. The Complete Generic Path

If you remember only one diagram, remember this:





The key distinction

A BSP is not just the kernel.

A practical BSP is the complete software needed to make a particular hardware platform usable:

```
BSP
=
Bootloader
+
Firmware
+
Kernel
+
Device Tree
+
Drivers
+
Root Filesystem
+
Toolchain/Sysroot
+
Board configuration
+
Flashing mechanism
+
Startup/services
```



And if you are developing a **custom hardware board**, the first bring-up can be even more hardware-heavy:

PCB
↓
Power rails
↓
Clock
↓
Reset
↓
Boot straps
↓
UART
↓
DDR
↓
Bootloader
↓
Kernel
↓
Device Tree
↓
Peripherals
↓
RootFS
↓
Application



So for **any new board**, your job is essentially to take it from **"silicon + PCB"** → **"Linux prompt"** → **"working peripherals"** → **"production application."**



Files, images, and data analysis are unavailable until usage resets at 8:18 PM. Continue chatting with text only, or upgrade for more access.

✦ [Try Plus free](#)

ChatGPT can make mistakes. Check important info.